Description du projet

En date du 28/08/2026, Rédigé par Mostoslavski David, DEVA 4A, Pour le module 321.

Historique des changements

- **11/09/2026** — Finition les sections vides (priorités, testing, CI/CD, intégration), clarification la politique de warnings et correction de quelques erreurs orthographiques.
- **28/08/2026** — Rédaction initiale.

Description générale de l'application

Une plateforme de streaming pour vidéo moyen à long format (20 min et 45 min).

En pratique

Un petit serveur avec moins de 10 vidéos de moyen-long format avec une vraie implémentation d'un service de streaming miniature.

Fonctionnalités

- Un système de login
- Un système de recherche par mots clés uniquement
- Une home/landing page avec simples recommandations
- Une implémentation de streaming simulée
- Gestion de la qualité de streaming.
- Sous-titres autogénérés avec light PML (pre-trained machine learning)
- ou pré-écrits au format `*.srt`
- Visionnage sans compte
- Favoris

landing page

- Vignettes style youtube
- barre de recherche (stub)
- Bouton "upload" (stub)
- bouton "account"
- filtre par tag (recherche de video)

Description de l'attribution des rôles

Les rôles sont attribués selon les préférences de chacun, et les besoins du groupe.

En classe DEVA 4A

• Adrien Wavelet • Timothee Delacrétaz • Dmytro Beznoskovych • David Mostoslavski

Les rôles

• Adrien : Dev (Front), Scrum master • Timothée : Dev, Responsable de la capture video des daily scrums • Dmytro : Dev, Responsable UI/UX, de la DX de l'API et du design de l'API (à l'exclusion de l'API gateway) • David : Product Owner, Dev Back, Planification architecturale

Description de la méthodologie de travail

Nous utilisons la méthodologie scrum agile avec quelques mineures modifications.

Daily reviews

Par “daily review” ou “daily scrum”, nous entendons réellement “un scrum bimensuel” afin de respecter les consignes du cours.

Les “daily reviews” sont enregistrées par vidéo, selon consignes du cours.

Backlogs

Le PO (David) est tenu de créer les stories principales dans le backlog. Les stories sont maintenues par Adrien (non-conventionnel à la méthodologie agile classique) Les stories peuvent être créées par les autres collaborateurs lorsqu’ils identifient une story importante qui n’est pas encore écrite, et ce, sans devoir attendre le prochain sprint, ni l’approbation du PO ou du scrum master.

Répartitions des stories

Chaque story est assignée au minimum à un collaborateur selon ses préférences et les besoins du projet.

Tout cela est discuté, au minimum une fois par semaine, en groupe et en dehors des vidéos des “daily reviews”.

Validation des stories / Définition de “tâche terminée”

Une tâche n’est pas nécessairement égale à “une branche git”.

- Les critères d’acceptation de la story sont satisfaits et vérifiés manuellement par l’auteur.
- une PR a été ouverte, relue et approuvée par une personne autre que l’auteur, puis mergée par le vérificateur.
- la CI passe au vert: build, tests unitaires, linter (les warnings ne bloquent pas — voir « Warning acceptance »)
- la documentation d’API est à jour si un endpoint a été ajouté, modifié ou supprimé (responsabilité de l’auteur, contrôle par Dmytro en review) ;
- aucune régression fonctionnelle constatée sur les fonctionnalités déjà livrées ;
- la carte est déplacée en Done par l’auteur

Definition des priorités

Les priorités sont définies par le PO et matérialisées par des labels GitHub :

- **P0 – Bloquant** : empêche le build, la CI ou une démo ; traité immédiatement.
- **P1 – Essentiel** : story nécessaire au MVP (login, recherche, streaming, landing page).
- **P2 – Souhaitable** : améliore l’expérience sans être indispensable (favoris, filtres par tag).
- **P3 – Optionnel / bonus** : sous-titres PML, gestion fine de la qualité, etc.

L’ordre du backlog reflète ces priorités ; en cas de conflit, le PO tranche.

Gestion du repository github

CI (continuous integration) La branche main est protégée contre: - Les PR non-validés par un collaborateur autre que l’auteur de celui-ci. - les “push” sans PR - les “force push”

Warning acceptance

Les warnings ne bloquent pas le développement ni la CI. Chaque auteur s'efforce de les résoudre avant la PR ; ceux qui subsistent sont discutés en review au cas par cas.

Documentation de l'API

- Exhaustive
- Pas nécessairement détaillée pour tous les services

Gestion des Pull-Requests

- Main branch sur github (protected).
- Une Dev branch
- Utilisation des conventional commits et branches
- PR obligatoire vers main ; push direct interdit sur main.
- Une seule lecture/validation par PR suffit
- L'auteur de la PR ne peut pas merge ses modifications lui-même

CI/CD

Définition du GitHub Actions

Pipeline GitHub Actions déclenché sur chaque PR vers `main` :

- **build** du/des service(s) concerné(s)
- **tests unitaires**
- **linter / formatage** (les warnings ne bloquent pas — voir « Warning acceptance »)

Le déploiement continu (CD) n'est pas dans le périmètre actuel du projet.

Qui gère les backlog, et comment

David, le PO.

GitHub Kanban

Testing

Le testing est utile et retenu, mais dosé selon la criticité :

- **Tests unitaires** obligatoires sur la logique métier (auth, recherche, gestion des favoris).
- Pas d'objectif de test exhaustif sur le code d'affichage / glue.

Code coverage ?

Le code coverage n'est pas strictement enforcé et ne fait pas échouer la CI. Il sert d'indicateur : on vise une couverture raisonnable de la logique métier, sans seuil bloquant.

Integration testing ?

Les tests d'intégration ne sont pas obligatoires mais recommandés aux frontières entre services (ex. gateway → service). Ils seront ajoutés au cas par cas si une régression inter-service apparaît.

Liste des outils de collaboration

- MS Teams (communication et daily scrum)
- Repo GitHub maintenu par David
- Kanban et Gantt intégrés au système de “Projet” et “issues” sur GitHub.
- L’outils de capture par défaut pour enregistrer les “daily scrums”

Lexique

PR := Pull Request MS := Microsoft GH := GitHub Daily Review := Daily Scrum CI := Continuous Integration CD := Continuous Deployement linter ~= formatage (infinitif) warning := Avertissement d’un LSP, linter, ou compiler qui ne bloque pas le développement, au contraire d’une “error/erreur”. PO := Product owner